

ROS 2 Python 환경 구성 가이드

ROS 2 워크스페이스와 일반 Python 프로젝트(uv 기반 가상환경)를 **한 머신에서 함께** 쓸 때 생기는 문제와 그 해결 방법을 정리한다. Ubuntu 24.04 / ROS 2 Jazzy 로의 마이그레이션 전략도 함께 다룬다.

- 대상 환경: Ubuntu 22.04 + ROS 2 Humble (Python 3.10) → Ubuntu 24.04 + ROS 2 Jazzy (Python 3.12)
- 점검 기준일: 2026-08-12

요약

질문	답
ROS 2 워크스페이스에 venv 를 써야 하나?	쓰지 않는다. 시스템 Python 을 그대로 쓰는 것이 정석이다
그럼 uv 는 버리나?	아니다. 비-ROS 프로젝트에서는 계속 uv 를 쓴다. 두 세계를 셸 단위로 분리하면 된다
가장 중요한 조치 한 가지는?	~/.bashrc 에서 ROS 자동 source 를 걷어내고 옵트인 (함수/ direnv)으로 바꾸는 것
apt 에 없는 Python 패키지가 필요하면?	--system-site-packages venv 를 만들되 activate 하지 않고 PYTHONPATH 끝에만 추가한다
22.04 → 24.04 업그레이드 방법은?	in-place 업그레이드 금지. 24.04 새로 설치 + Humble 은 Docker 로 유지

1. venv 가 ROS 2 에서 깨지는 이유

원인은 세 가지이고, 셋 다 pip/venv 의 규칙이 아니라 **ROS 가 환경변수로 모듈 경로를 주입한다**는 데서 나온다. 이 셋을 이해하면 이후의 모든 권고가 따라 나온다.

(a) rclpy 는 시스템 인터프리터에 컴파일된 C 확장이다

rclpy 는 순수 Python 이 아니라 바이너리 확장 모듈을 포함한다.

```
/opt/ros/humble/lib/python3.10/site-packages/rclpy/_rclpy_pybind11.cpython-310-x86_64-linux-gnu.so
```

파일명의 cpython-310 이 말해 주듯 **Python minor 버전이 정확히 일치해야 import 된다.** uv 로 venv 를 만들면 기본값은 uv 가 내려받은 최신 standalone Python(3.13/3.14 등)이므로 import rclpy 자체가 실패한다.

ROS 배포판별 Python 버전은 **고정이며 선택 사항이 아니다.**

배포판	Python	배포판	Python
Humble (22.04)	3.10	Jazzy (24.04)	3.12

(b) PYTHONPATH 는 venv 의 site-packages 보다 우선한다

`source /opt/ros/humble/setup.bash` 와 `source install/setup.bash` 는 `PYTHONPATH` 에 경로를 밀어넣는다. 실제 값은 다음과 같다.

```
~/development/ros/rdfp_ws/install/rdfp/lib/python3.10/site-packages
~/development/ros/rdfp_ws/install/rdfp_msgs/local/lib/python3.10/dist-packages
/opt/ros/humble/lib/python3.10/site-packages
/opt/ros/humble/local/lib/python3.10/dist-packages
```

CPython 의 `sys.path` 구성 순서는 이렇다.

```
''(스크립트 디렉터리) → PYTHONPATH → stdlib → venv site-packages
→ 시스템 dist-packages (--system-site-packages 인 경우)
```

즉 **venv 를 활성화해도 ROS 경로가 항상 앞선다**. venv 가 ROS 를 격리하지 못한다는 뜻이고, 반대로 ROS 를 source 한 셀에서는 venv 가 온전히 격리되지도 않는다.

이 순서에는 중요한 따름정리가 하나 있다 — **PYTHONPATH 에 넣은 경로는 시스템 dist-packages 보다 항상 앞선다**. 그래서 venv 에 `numpy` 를 깔고 그 경로를 `PYTHONPATH` 에 (끝에라도) 추가하면, apt 가 설치한 `python3-numpy` 를 가려 버린다. 4-B 절의 주의사항이 여기서 나온다.

(c) ros2 run 으로 띄운 노드는 venv 를 아예 보지 않는다

`colcon build` 가 생성하는 console script 의 shebang 은 빌드에 쓰인 인터프리터의 절대 경로로 박힌다.

```
$ head -1 install/rdfp/lib/rdfp/camera_node
#!/usr/bin/python3
```

절대 경로이므로 `PATH` 를 무시한다. venv 를 activate 한 셀에서 `ros2 run rdfp camera_node` 를 실행해도 노드는 시스템 Python 으로 실행되고, venv 에 `pip install` 한 라이브러리는 `ModuleNotFoundError` 가 난다.

이것이 "venv 를 만들었더니 되다 안 되다 한다" 의 정체다. `python -m` 으로 직접 실행할 때는 되고 `ros2 run` / `ros2 launch` 로는 안 되기 때문에 원인을 찾기 어렵다.

결론: ROS 워크스페이스에서 venv 는 격리도 못 하고(a, b) 적용도 안 된다(c). 시스템 Python 을 쓰는 것이 회피가 아니라 정석이다.

2. 셀 환경 구성 — 위험 지점과 적용된 조치

2.1 자동 source 는 모든 셀을 오염시킨다

과거 `~/.bashrc` 는 ROS 환경을 무조건 적용했다.

```
# ~/.bashrc (이전)
source $HOME/.ros2rc      # 배포판 공통
source $HOME/.rdfprc      # 워크스페이스 overlay + cd $RDFP_HOME
```

결과적으로 모든 셀에 PYTHONPATH · LD_LIBRARY_PATH · AMENT_PREFIX_PATH 가 박혔다. 그 상태로 비-ROS 프로젝트에서 .venv 를 활성화하면, 1-(b) 에 따라 /opt/ros/humble/.../site-packages 가 sys.path 최상단에 그대로 남는다. ROS 와 전혀 무관한 프로젝트에서 원인 불명의 import 충돌이 나는 전형적인 경로다. LD_LIBRARY_PATH 도 마찬가지로 비-ROS 바이너리에 ROS 의 공유 라이브러리를 먹인다.

조치 — 자동 source 를 걷어내고 옵트인(함수 정의)으로 바꾼다. source 는 즉시 실행이지만 함수 정의는 이름만 등록하므로, 셀을 열어도 환경이 바뀌지 않는다. 적용된 구성은 2.3 절에 있다.

2.2 ~/.local 의 pip 패키지가 apt 패키지를 가리고 있다

```
$ python3 -c "import numpy; print(numpy.__version__, numpy.__file__)"
1.26.4 /home/kwlee/.local/lib/python3.10/site-packages/numpy/__init__.py
```

user site (~/.local/lib/python3.10/site-packages) 는 시스템 dist-packages 보다 우선하므로, **ROS 노드가 실제로 쓰는 numpy 도 이쪽이다.**

현재 1.26 은 numpy 1.x ABI 라서 문제가 없다. 그러나 pip install --user -U numpy 등으로 **2.x 가 올라가는 순간** cv_bridge 를 비롯해 numpy 1.x 헤더로 빌드된 ROS C 확장이 전부 깨진다. 증상은 다음과 같은 형태로 나타난다.

```
A module that was compiled using NumPy 1.x cannot be run in NumPy 2.x
```

해결 — ~/.local 에 ROS 가 이미 제공하는 패키지를 두지 않는다. 특히 numpy , PyYAML , opencv-python , lark , setuptools . 이미 들어가 있다면 제거하고 apt 버전으로 되돌리는 편이 안전하다.

```
python3 -m pip list --user # 무엇이 들어와 있는지 먼저 확인
```

2.3 적용된 구성 — 2단 rc + 옵트인 함수

현재 이 머신에 적용된 구성이다. 배포판 공통 층과 워크스페이스 층을 분리하고, 둘 다 자동 source 하지 않는다.

```
~/.bashrc
# 함수 "정의"만 — 셀 환경을 건드리지 않는다
├─ rosenv [distro] → ~/development/ros/.ros2rc      배포판 공통
└─ rdfpenv [distro] → rdfp_ws/.rdfprc              워크스페이스 overlay
    └─ source ~/.ros2rc
```

명령	하는 일
(없음)	깨끗한 셀. PYTHONPATH · AMENT_PREFIX_PATH 비어 있음 — uv 프로젝트용
rosenv	ROS 배포판 환경만. 배포판은 OS 버전에서 자동 판별

명령	하는 일
<code>roscv jazzy</code>	배포판 명시
<code>rdcpenv</code>	위 + <code>RDFP_*</code> 변수 + <code>install/setup.bash</code> + 워크스페이스로 <code>cd</code>

ROS 환경이 켜지면 프롬프트 앞에 `[ros:humble]` 이 붙어 어느 배포판 셸인지 눈으로 구분된다. 두 배포판이 공존하는 마이그레이션 기간에 특히 중요하다.

배포판 자동 판별

`.ros2rc` 를 배포판마다 고쳐 쓰지 않도록, 인자가 없으면 OS 버전에서 판별한다.
같은 파일을 22.04 머신·24.04 머신·Docker 컨테이너에서 그대로 쓸 수 있다
(컨테이너 안의 `/etc/os-release` 는 컨테이너 베이스 이미지 것이므로, 24.04 호스트에서 Humble 컨테이너를 띄워도 그 안에서는 `humble` 로 판별된다).

우선순위는 다음과 같다.

순위	출처	없을 때
1	함수 인자 — <code>roscv jazzy</code>	—
2	<code>ROS2_DEFAULT_DISTRO</code> 환경변수	—
3	OS 버전 매핑 — 22.04 → <code>humble</code> , 24.04 → <code>jazzy</code>	4 로
4	<code>/opt/ros/*</code> 스캔 — 설치본이 하나뿐이면 그것	에러

1·2 는 명시 지정이므로 풀백하지 않는다. 요청한 배포판이 설치돼 있지 않으면
에러로 끝난다 — 사용자가 지정한 것과 다른 배포판이 조용히 켜지면 안 되기 때문이다.
3·4 는 자동 판별이므로 서로 흡수한다. 설치본이 둘 이상인데 3 이 실패하면 모호하므로
에러를 내고 명시 지정을 요구한다.

```
$ roscv                # 22.04 → humble 자동
$ roscv jazzy          # 미설치 시
roscv: ROS 2 'jazzy' not found at /opt/ros/jazzy
```

`/etc/os-release` 는 서브셸에서 읽으므로 `NAME / VERSION_ID` 가 셸에 남지 않는다.

`~/development/ros/.ros2rc` 가 갖는 방어 장치는 다음과 같다.

장치	이유
중복 호출 시 조기 <code>return</code>	재호출로 <code>PATH / PYTHONPATH</code> 가 계속 길어지는 것을 막는다
다른 배포판이 이미 켜져 있으면 거부	Humble/Jazzy 경로가 한 셸에 뒤섞이면 진단이 불가능해진다
배포판별 <code>ROS_DOMAIN_ID</code> (<code>humble 31 / jazzy 32</code>)	두 배포판을 동시에 띄웠을 때 서로를 <code>discovery</code> 하지 않게 한다. 배포판 간에는 메시지 정의가 달라 정상 통신이 안 되면서 <code>ros2 topic list</code> 에 남의 토픽이 섞여 보이는, 진단하기 까다로운 증상이 생긴다
<code>librmw_cyclonedds_cpp.so</code> 존재 확인 후 <code>RMW_IMPLEMENTATION</code> 설정	해당 배포판에 미설치면 노드가 RMW 로드 실패로 아예 뜨지 않는다
<code>colcon_argcomplete hook -f</code> 가드	없는 환경에서 에러를 내지 않는다

LD_LIBRARY_PATH 수동 설정은 제거했다. `setup.bash` 가 이미 올바르게 설정하므로 중복이고, `source` 할 때마다 앞에 덧붙여 길어지며, 비-ROS 바이너리에 ROS 공유 라이브러리를 먹이는 위험만 키운다.

```
# 제거된 줄
export LD_LIBRARY_PATH=/opt/ros/humble/lib:$LD_LIBRARY_PATH
```

.rdfpnc 는 **RDFP_DB_DSN** 에 평문 비밀번호를 포함한다. 워크스페이스 안에 있고 `git` 에 추적되지 않으므로 `.gitignore` 에 `./rdfpnc` 를 등록해 `git add -A` 사고를 막았다.

direnv 대안 — 디렉터리 진입 시 자동 적용하고 나갈 때 자동 해제하는 방식이다. 디렉터리 단위 옵트인이라 더 정확하지만, `alias`/함수는 환경변수가 아니라서 전달되지 않는다(`cc alias` 는 살아남지 않는다). 현재 이 머신에는 설치되어 있지 않다.

```
sudo apt install direnv          # + ~/.bashrc 에 eval "$(direnv hook bash)"
echo 'source ./rdfpnc' > .envrc && direnv allow
```

3. 권장 구성 — 원칙 네 가지

1. **ROS 워크스페이스는 시스템 Python 만 쓴다.** `venv` 를 만들지 않는다. `rc1py` , 메시지 패키지, `colcon` 은 `apt` 가 관리한다.
2. **ROS 노드가 쓸 추가 의존성은 `apt( python3-<pkg> )` 우선.** `rosdep` 이 해결해 주고, `package.xml` 에 선언할 수 있고, 다른 머신에서도 재현된다.
3. **`apt` 에 없는 순수 Python 패키지만 예외적으로 `venv` 에 넣고, `activate` 하지 않고** 경로만 `PYTHONPATH` 에 추가한다 (4-B).
4. **비-ROS 프로젝트는 지금처럼 `uv` 로 간다.** 단 ROS 를 `source` 하지 않은 셸에서만 쓴다 (2.1).

4. ROS 2 프로젝트를 만드는 절차

A. 기본형 — venv 없음 (거의 모든 경우 이것을 쓴다)

```
# 1) 워크스페이스 생성
mkdir -p ~/development/ros/<ws>/src && cd ~/development/ros/<ws>

# 2) ROS 환경 진입 (자동 source 를 켜다면)
rosv

# 3) 패키지 생성
cd src
ros2 pkg create --build-type ament_python --license Apache-2.0 my_pkg

# 4) 의존성은 package.xml 에 선언한 뒤 rosdep 으로 설치
#   <exec_depend>python3-numpy</exec_depend>
cd ~/development/ros/<ws>
rosdep install --from-paths src --ignore-src -r -y

# 5) 빌드 — 반드시 워크스페이스 루트에서 실행한다
colcon build --packages-select my_pkg
source install/setup.bash
```

src/ 안에서 colcon build 를 실행하면 src/<pkg>/install · src/<pkg>/build 가 생겨 실제 워크스페이스 install 을 PYTHONPATH 로 가린다. 상세는 워크스페이스 [CLAUDE.md](#) 의 "Important non-obvious behaviors" 참고.

에디터(VS Code) 설정 — venv 없이 자동완성을 살리는 방법이다. 인터프리터를 /usr/bin/python3 로 지정하고 .vscode/settings.json 에 ROS 경로를 추가한다.

```
{
  "python.defaultInterpreterPath": "/usr/bin/python3",
  "python.analysis.extraPaths": [
    "/opt/ros/humble/lib/python3.10/site-packages",
    "${workspaceFolder}/install/my_pkg/lib/python3.10/site-packages"
  ]
}
```

IDE 때문에 venv 를 만드는 경우가 많은데, 이 설정으로 충분하다.

B. 예외형 — apt 에 없는 패키지가 필요할 때

핵심은 venv 를 "활성화"하는 게 아니라 "경로만 빌려온다" 는 것이다. 1-(c) 의 shebang 문제 때문에 activate 는 어차피 의미가 없다.

```
# 시스템 인터프리터로 venv 생성 - --python 을 반드시 명시한다
uv venv --python /usr/bin/python3 --system-site-packages .venv

# activate 하지 않고 그 인터프리터를 지정해 설치
uv pip install --python .venv/bin/python <apt에-없는-패키지>

# 경로만 PYTHONPATH 끝에 추가한다
export PYTHONPATH="$PYTHONPATH:$PWD/.venv/lib/python3.10/site-packages"
```

주의점 두 가지:

- 이 venv 에는 ROS 가 이미 제공하는 패키지를 절대 넣지 않는다 (numpy , PyYAML , opencv-python , lark ...). 1-(b) 의 따름정리대로, PYTHONPATH 끝에 붙여도 시스템 dist-packages 보다는 앞서기 때문에 apt 버전을 가려 버린다.
- python3.10 은 배포판에 따라 달라진다(Jazzy 는 python3.12). 하드코딩하지 말고 워크스페이스 셋업 스크립트에서 계산하는 편이 낫다.

```
PYVER=$(python3 -c 'import sys; print(f"python{sys.version_info.major}.{sys.version_info.minor}")')
export PYTHONPATH="$PYTHONPATH:$PWD/.venv/lib/$PYVER/site-packages"
```

C. 완전 격리가 필요하다면 — pixi + RoboStack (참고)

여러 배포판(Humble/Jazzy)을 한 머신에서 오가야 하거나 정말로 venv 처럼 잠긴 환경을 원한다면, apt ROS 대신 **RoboStack**(conda-forge 계열 채널의 ros-humble-* / ros-jazzy-* 패키지)을 pixi 로 쓰는 방법이 있다. ROS 와 pip 의존성이 하나의 lock 파일로 관리되고 프로젝트별로 배포판이 달라도 된다.

대가는 분명하다 — apt 생태계에서 벗어나므로 벤더 드라이버나 일부 MoveIt 리소스 패키지가 없을 수 있고, 튜토리얼·에러 검색 결과가 맞지 않는다. **이 워크스페이스는 moveit_resources_panda 등 apt 패키지와 하드웨어 스택에 묶여 있으므로 권하지 않는다.** 선택지로만 알아 둔다.

5. 트러블슈팅

증상	원인	조치
venv 에서 ModuleNotFoundError: rclpy	venv 의 Python minor 버전이 ROS 와 다름 (1-a)	venv 를 쓰지 않거나, --python /usr/bin/python3 로 재생성
python 으로는 되는데 ros2 run 에서 ModuleNotFoundError	console script shebang 이 시스템 Python (1-c)	4-B 의 PYTHONPATH 방식으로 전환
비-ROS 프로젝트에서 엉뚱한 패키지 버전이 잡힘	셸에 ROS 환경이 켜져 있음 (2.1)	echo \$ROS_DISTRO 로 확인. 켜져 있으면 새 셸에서 작업 (2.3)

증상	원인	조치
ros2 명령을 못 찾음	옵트인이라 기본 셸에는 ROS 가 없다	rosvnenv 또는 rdfsenv (2.3)
A module that was compiled using NumPy 1.x...	pip numpy 2.x 가 apt numpy 를 가림 (2.2)	pip uninstall --user numpy 후 apt 버전 사용
코드를 고쳤는데 반영이 안 됨	src/ 하위의 stale install / build 가 shadowing	find src -maxdepth 2 \(-name install -o -name build \) 후 제거
ros2 launch 가 설정 YAML 을 못 찾음	패키지 share 에는 config/* 만 설치됨	src/<pkg>/config/ 에 두거나 절대경로 인자 전달

6. Ubuntu 24.04 / ROS 2 Jazzy 마이그레이션

6.1 달라지는 것

항목	22.04 / Humble	24.04 / Jazzy	영향
Python	3.10	3.12	install/.../python3.10/... 경로가 전부 바뀐다. 하드코딩된 곳 확인 필요
distutils	stdlib 포함	stdlib 에서 제거	from distutils... 사용처가 있으면 깨진다 (setuptools shim 에 의존하게 됨)
pip 정책	제한 없음	PEP 668 externally-managed	시스템 Python 에 pip install 이 거부된다
numpy (apt)	1.21	1.26	여기서 pip 으로 2.x 를 올리면 C 확장이 전멸한다
setuptools (apt)	59	68	시스템 setuptools 를 pip 으로 업그레이드하지 말 것. setup.py develop 이 제거되면 colcon build --symlink-install 이 깨진다

PEP 668 은 오히려 좋은 소식이다 — 24.04 에서는 2.2 절의 사고(실수로 ~/.local 에 numpy 를 덮어쓰는 것)가 기본적으로 차단된다. 다만 --break-system-packages 를 습관적으로 쓰기 시작하면 그 방어가 무력화되므로, 쓰지 않는 것을 원칙으로 삼는다.

6.2 이행 전략

do-release-upgrade 를 통한 in-place 업그레이드는 권하지 않는다. ROS apt 저장소 교체, Python 3.10 ↔ 3.12 혼재, ~/.local 잔재가 겹쳐 복구가 어려운 상태가 되기 쉽다.

권장은 **24.04** 를 새로 설치하고 Humble 은 Docker 컨테이너로 남기는 것이다. 이 워크스페이스는 이미 Docker 구성을 갖고 있어([docker/README.md](#)) 진입 장벽이 낮다.

1. 24.04 + Jazzy 를 호스트에 네이티브 설치 — 새 개발은 여기서 한다.
2. Humble 은 osrf/ros:humble-desktop 기반 컨테이너에 워크스페이스를 볼륨 마운트하여 회귀 검증용으로 유지한다. RViz 등 GUI 는 X11 소켓 마운트로 해결된다.
3. 두 환경에서 같은 소스를 빌드해 가며 하나씩 포팅한다. **롤백 지점이 항상 존재하는 것이 이 방식의 핵심 장점이다.**

6.3 포팅 전에 확인할 것

- `moveit_resources_panda / moveit_resources_panda_moveit_config` 의 **Jazzy 바이너리 존재 여부** — 이 워크스페이스의 launch 전체가 여기에 의존한다. 없으면 소스 빌드가 필요하며, 이것이 마이그레이션 최대 리스크다.
- `rmw_cyclonedds_cpp` (`~/ros2rc` 에서 지정 중) 의 Jazzy 패키지명과 기본 설정 변화.
- `moveit_servo` 파라미터 스키마 변경 — Humble ↔ Jazzy 사이에 파라미터가 꽤 움직인 편이라 별도 검증이 필요하다.
- `sensor_msgs / control_msgs` 등 메시지 정의 변경 여부.
- `setup.py` 의 `distutils` 직접 사용 여부.

관련 문서

- [../CLAUDE.md](#) — 빌드 명령, "Important non-obvious behaviors" (stale build artifact shadowing 등 환경 관련 함정)
- [../docker/README.md](#) — Docker 실행 구성. apt ROS 2 만 사용해 소스빌드 MoveIt 과의 ABI 충돌을 회피하는 근거
- [../src/rdfp/CLAUDE.md](#) — apt/pip 의존성 분리 규칙, console script 추가 절차
- [../INDEX.md](#) — 저장소 전체 문서 인덱스